

Memoria del proyecto

API de gestión de gimnasio (FastAPI) y aplicación web (React)

Repositorio: github.com/IA-P1-BCN/g2-python-crud-api

Equipo: Eva, Nayeli, Pedro, Ivanna y Elena

Organización: IA-P1-BCN

Fecha: 2 de octubre de 2026

Índice

1. Resumen ejecutivo
2. Objetivo y alcance
3. Stack tecnológico
4. Arquitectura del frontend
5. Cómo encajan front y back
6. Estrategia de pruebas
7. Integración continua y despliegue
8. Backlog del producto
9. Reparto del equipo
10. Gestión del repositorio
11. Estado actual y próximos pasos
12. Anexo: comandos útiles

1. Resumen ejecutivo

El proyecto consiste en una **API REST de gestión de gimnasio** desarrollada con FastAPI y una **aplicación web** en React que la consume. El trabajo se organiza en cuatro fases de dificultad creciente (Esencial, Medio, Avanzado y Experto) que cubren desde el CRUD y las reglas de negocio hasta autenticación JWT, caché, WebSockets, Docker, CI/CD y despliegue en la nube.

En esta primera entrega se ha dejado preparado el **andamiaje del frontend** (estructura, capa de API, autenticación y enrutado por roles) y se ha generado el **backlog completo** en GitHub Projects, con historias de usuario, subtareas, prioridades, estimaciones y responsables.

28

historias de usuario

95

subtareas

4

fases / milestones

5

personas

El repositorio trabaja con dos ramas: `main` (vacía, reservada) y `dev` (rama por defecto y de integración). Todo el desarrollo se hace en ramas de característica y se integra por pull request.

2. Objetivo y alcance

Objetivo

Construir una plataforma que permita gestionar socios, entrenadores, planes de membresía, clases, horarios, reservas y pagos, aplicando buenas prácticas de desarrollo: separación en capas, pruebas automatizadas, integración continua y despliegue reproducible.

Alcance funcional

- Gestión de usuarios con roles (administrador, entrenador y socio).
- Planes de membresía y asignación de suscripciones con fechas de validez.
- Clases, horarios y control de aforo.
- Reservas y cancelaciones con reglas de negocio (membresía activa, aforo, solapamiento).
- Pagos asociados a membresías.
- Documentación interactiva (Swagger), filtros, paginación y exportación a CSV.
- Autenticación JWT, permisos por rol, caché y disponibilidad en tiempo real.
- Interfaz web básica para socios.

Alcance de esta entrega

- Estructura del proyecto frontend React + Vite + TypeScript.
- Backlog completo creado en GitHub Projects con reparto asignado.
- Pipeline de CI con dos jobs (backend y frontend).

3. Stack tecnológico

Frontend

- React 18 + TypeScript
- Vite como empaquetador
- React Router (enrutado por rol)
- TanStack Query (estado del servidor)
- axios (cliente HTTP)

Backend

- Python + FastAPI
- SQLAlchemy + Alembic
- PostgreSQL
- Pydantic (validación)
- JWT (autenticación)

Calidad y pruebas

- Vitest + Testing Library (unitarios)
- Playwright (end-to-end)
- pytest (backend)
- ESLint + Prettier

Infraestructura

- Docker + nginx
- GitHub Actions (CI)
- Render / Railway (API y BD)
- Vercel / Netlify (frontend)

4. Arquitectura del frontend

4.1 Estructura de carpetas

```
frontend/
├── src/
│   ├── api/           # ÚNICO sitio que llama a la API (axios + interceptores)
│   ├── auth/          # AuthContext, ProtectedRoute, tokenStorage, roles
│   ├── components/ui/  # Button, Table, Pagination, Alert reutilizables
│   ├── layouts/        # AppLayout (cabecera + navegación por rol)
│   ├── pages/
│   │   ├── member/    # Pantallas del socio
│   │   ├── trainer/   # Pantallas del entrenador (HU-29)
│   │   └── admin/     # Pantallas del admin, planes (HU-28)
│   ├── types/         # schema.d.ts generado desde OpenAPI
│   └── lib/            # queryClient
├── tests/
│   ├── unit/          # Vitest
│   └── e2e/           # Playwright
├── Dockerfile · nginx.conf
└── package.json · vite.config.ts · playwright.config.ts
```

4.2 Capa de API

El módulo `src/api/` es el único punto que habla con el backend. El cliente de axios añade el token en cada petición y, si el servidor responde `401`, borra la sesión y redirige al login. Los errores se normalizan con `toApiError` para mostrarlos de forma uniforme.

Si el backend cambia, solo se toca esta capa; las páginas no conocen detalles de HTTP.

4.3 Autenticación y roles

- **AuthContext**: guarda el usuario y su rol; expone `login`, `logout` y `hasRole`.
- **tokenStorage**: único punto de acceso a la sesión persistida.
- **ProtectedRoute**: bloquea rutas sin sesión (redirige a login) y por rol (403).

4.4 Componentes UI reutilizables

Se incluyen `Button`, `Table`, `Pagination` y `Alert`, pensados para que cada persona los reutilice en sus pantallas y mantenga una interfaz coherente.

4.5 Páginas por rol

Las pantallas de dominio quedan como **stubs** con su propietario y su historia asignada, listas para que cada miembro del equipo implemente su parte:

Pantalla	Responsable	Historia
Reservas y formulario de reserva (socio)	P5 · Elena	Socios
Listado de socios (admin)	P5 · Elena	Socios
Gestión de planes (admin)	P2 · Pedro	HU-28
Socios del entrenador	P3 · Nayeli	HU-29
Pruebas end-to-end	P4 · Eva	HU-30

5. Cómo encajan front y back

Generación de tipos

FastAPI publica su esquema en `/openapi.json`. Con `npm run api:types` se regenera `src/types/schema.d.ts` mediante `openapi-typescript`. Ese fichero no se edita a mano: si el backend cambia un campo, el front deja de compilar y el contrato roto se detecta en CI.

CORS

En desarrollo, Vite hace proxy de `/api` hacia `http://localhost:8000`, de modo que el navegador nunca cruza orígenes. En producción, el front llama directamente a la URL del backend y FastAPI debe permitir el origen del front en su middleware CORS.

Dónde se guarda el token

El token se guarda en `localStorage` y viaja como cabecera `Authorization: Bearer`. Al recargar, la sesión se recupera del almacenamiento.

Errores de la API

El interceptor de respuesta detecta el `401` y cierra sesión. El resto de errores se muestran con `<Alert variant="error">` a partir del mensaje normalizado.

La seguridad real está en el backend

Ocultar rutas con `ProtectedRoute` es solo experiencia de usuario: cualquiera puede modificar el estado en el navegador. La autorización de verdad se valida en el backend en cada endpoint, comprobando el token y el rol.

6. Estrategia de pruebas

Se cubren los tres niveles que pide la rúbrica: unitario, integración y end-to-end.

Nivel	Herramienta	Qué cubre
Unitario	Vitest + Testing Library	ProtectedRoute (roles), Pagination, cliente de API (token y errores)
Integración	pytest + FastAPI TestClient	Flujo completo de reservas, reglas de negocio y relaciones
End-to-end	Playwright	Login, reservar, cancelar y acceso denegado por rol

En el frontend hay actualmente **9 pruebas unitarias** (3 ficheros) y un `example.spec.ts` de humo en E2E, que sirve de base para que P4 escriba los flujos reales.

7. Integración continua y despliegue

Integración continua

El flujo de GitHub Actions tiene **dos jobs** independientes:

- **Backend**: instalación de dependencias, lint con ruff y tests con pytest.
- **Frontend**: `npm ci`, lint, formato, tests unitarios, build (typecheck) y tests E2E con Playwright.

Contenedor y servidor web

El frontend se empaqueta con un **Dockerfile multi-etapa** (build con Node y servido con nginx). El `nginx.conf` aplica el fallback de SPA hacia `index.html` y cachea los assets con hash.

Despliegue

- API y base de datos: **Render** o **Railway**.
- Frontend: **Vercel** o **Netlify**.
- La URL de la API se inyecta en tiempo de build con `VITE_API_URL`.

8. Backlog del producto

El backlog está creado en GitHub Projects (proyecto **#28**) y se estructura en cuatro fases. Cada historia incluye criterios de aceptación y cada subtarea está enlazada a su historia como *sub-issue*.

Fase	Historias	Subtareas	Contenido
1 · Esencial	12	42	Setup, BD y diagrama ER, CRUDs, reservas con reglas de negocio, excepciones, tests, README y Kanban
2 · Medio	7	19	Swagger, errores HTTP, filtros y paginación, CSV, endpoints de relación, pagos, tests de integración
3 · Avanzado	4	16	JWT, roles y permisos (RBAC), caché, WebSockets
4 · Experto	5	18	Docker, GitHub Actions, despliegue en la nube, Stripe en modo test, interfaz React
Total	28	95	

Metadatos de las issues

- **Labels:** nivel (essential/medium/advanced/expert), tipo (user-story/subtask) y área (backend, database, auth, testing, docs, devops, scrum, frontend, performance, realtime).
- **Issue type:** `Feature` para historias y `Task` para subtareas.
- **Priority:** P0 (imprescindible), P1 (deseable), P2 (opcional).
- **Estimate:** 1 (½ día), 2 (1 día), 3 (2 días), 4 (3 días o más).
- **Milestones:** una por fase, para agrupar la vista del proyecto.

El backlog se genera de forma reproducible con el script `scripts/create_backlog.sh`, configurable por variables de entorno.

9. Reparto del equipo

El reparto se ha equilibrado por carga (23–25 issues por persona) minimizando el número de historias movidas. Cada persona recibe historias completas junto con sus subtareas.

Persona	Usuario	Issues	Historias asignadas
Eva	<code>miskybox</code>	23	Setup, Classes CRUD, Kanban/SCRUM, JWT, CI
Nayeli	<code>nagicome03</code>	25	Database design, Class schedules, Swagger, Payments, CSV export, RBAC
Pedro	<code>PDguezPgr</code>	25	Users CRUD, Bookings rules, Filtros/paginación, Caché, Cloud deployment
Ivanna	<code>IvannaRCA</code>	25	Plans CRUD, Excepciones, Errores avanzados, Integration tests, Docker, Stripe
Elena	<code>elenaalmansacampos</code>	25	Memberships, Test infra, Docs, Relationship endpoints, WebSockets, Web UI

Ajuste de equilibrio

Para equilibrar las cargas se movieron cuatro historias (con sus subtareas):

- CSV export: de Pedro a Nayeli.
- Stripe (pagos): de Nayeli a Ivanna.
- Documentación Markdown: de Ivanna a Elena.
- Relationship endpoints: de Eva a Elena.

10. Gestión del repositorio

- **Rama por defecto:** `dev`, usada para integrar el trabajo del equipo.
- **Rama `main`:** reservada y vacía, destinada a versiones estables.
- **Ramas de característica:** `feature/*` y `fix/*`.
- **Integración:** mediante pull request con al menos una revisión y CI en verde.
- **Scripts:** `scripts/create_backlog.sh` genera el backlog de forma reproducible.

Documentación del frontend

El `README.md` del frontend incluye la sección «Cómo encajan front y back» con generación de tipos, CORS, gestión del token, errores y el principio de que la seguridad real reside en el backend.

11. Estado actual y próximos pasos

Completado

- Estructura del frontend React + Vite + TypeScript con capa de API, autenticación y rutas por rol.
- Componentes UI reutilizables y pantallas de dominio como stubs asignados.
- Pruebas unitarias del andamiaje y ejemplo E2E.
- Dockerfile, nginx y pipeline de CI con dos jobs.
- Backlog completo en GitHub Projects con reparto equilibrado.
- Ramas `dev` / `main` publicadas, con `dev` por defecto.

Próximos pasos

1. Implementar las pantallas de cada rol a partir de los stubs (P5, P2, P3).
2. Escribir los tests E2E reales de login, reserva, cancelación y acceso por rol (P4).
3. Desarrollar el backend siguiendo las fases del backlog.
4. Configurar despliegue en Render/Railway y Vercel/Netlify.
5. Revisar prioridades y estimaciones en el sprint planning.

Documento generado automáticamente a partir del estado del repositorio y del proyecto en GitHub. Gym CRUD · IA-P1-BCN · 2 de octubre de 2026.

Anexo: comandos útiles

Frontend

Comando	Descripción
<code>npm install</code>	Instala dependencias
<code>npm run dev</code>	Servidor de desarrollo (proxy /api → :8000)
<code>npm run build</code>	Typecheck + build de producción
<code>npm run test</code>	Tests unitarios (Vitest)
<code>npm run test:e2e</code>	Tests end-to-end (Playwright)
<code>npm run lint / format</code>	ESLint / Prettier
<code>npm run api:types</code>	Regenera tipos desde OpenAPI

Backlog

Comando	Descripción
<code>bash scripts/create_backlog.sh</code>	Genera el backlog en GitHub Projects
<code>REPO=... PROJECT_NUMBER=... bash ...</code>	Ejecuta contra otro repo/proyecto
<code>SKIP_ASSIGNEES=1 bash ...</code>	Crea el backlog sin asignar responsables

Enlaces

- Repositorio: github.com/IA-P1-BCN/g2-python-crud-api
- Proyecto: github.com/orgs/IA-P1-BCN/projects/28